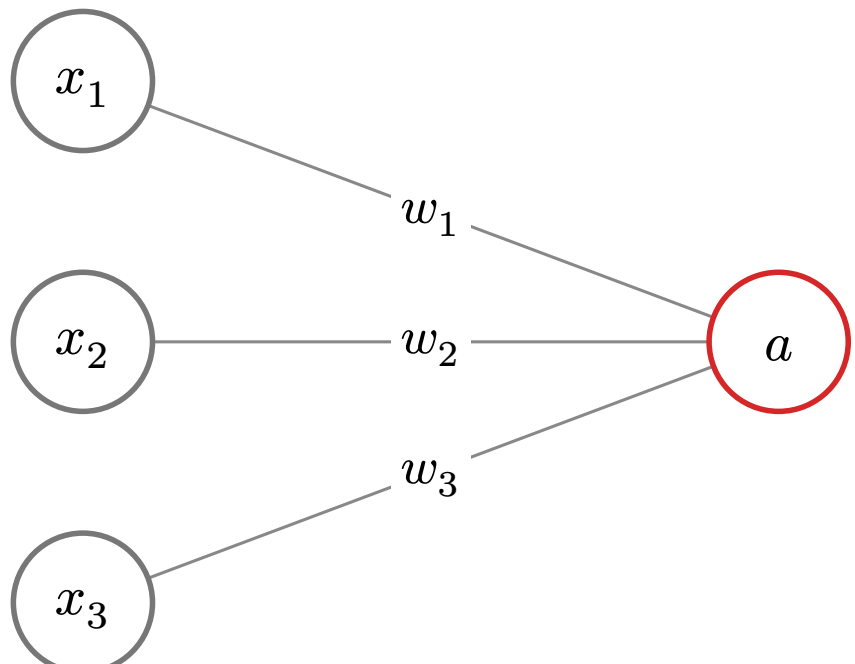
Redes neuronales

Tomas Santana

November, 2026

Una neurona

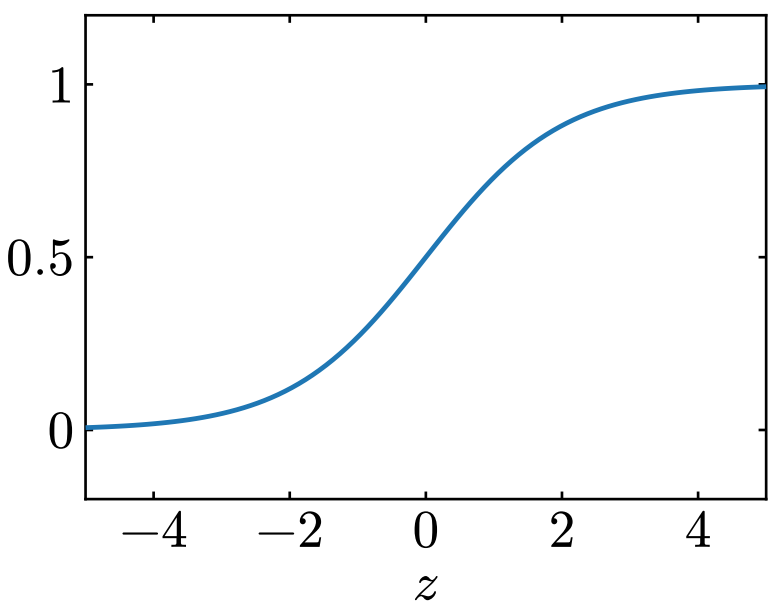


entrada

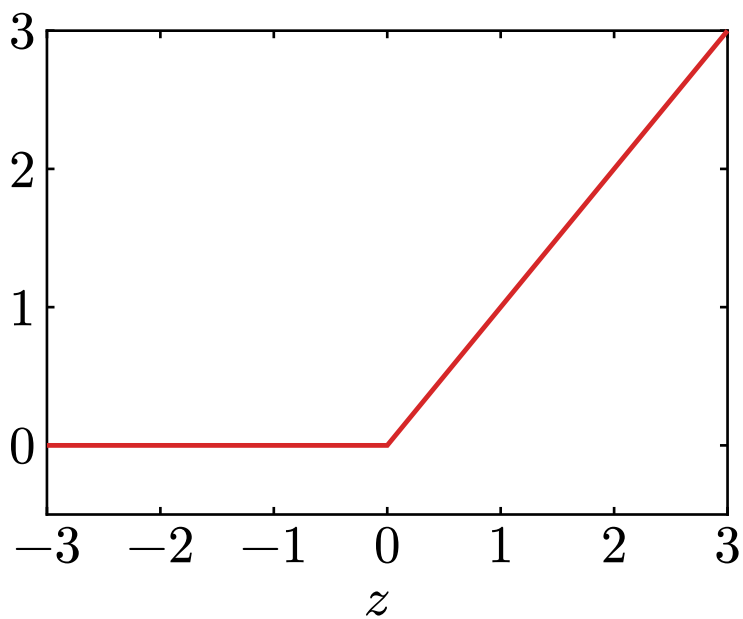
$$a = g(\mathbf{w}^T \mathbf{x} + b)$$

Funciones de activación

Sigmoide $g(z)$

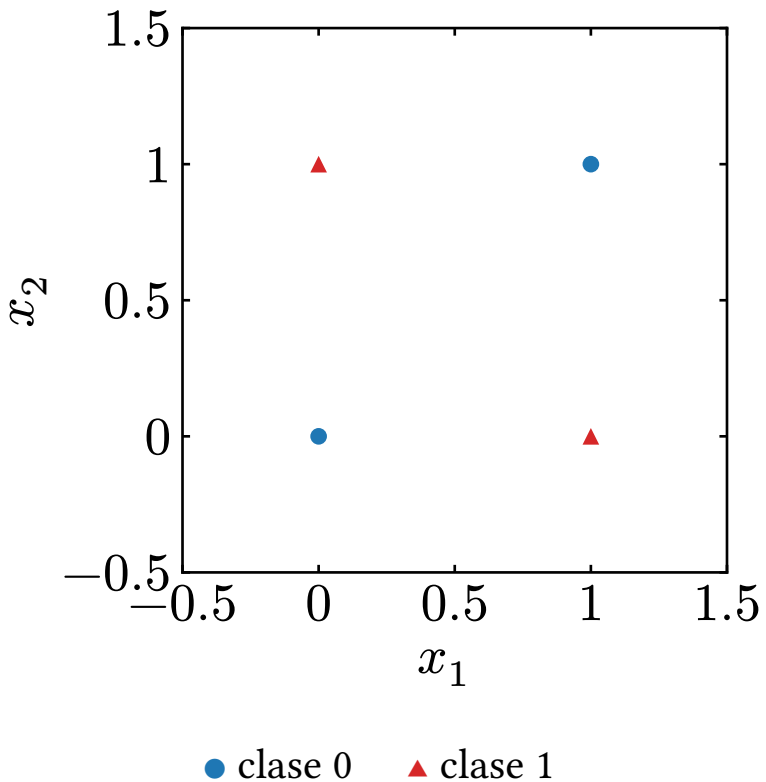


ReLU $\max(z, 0)$

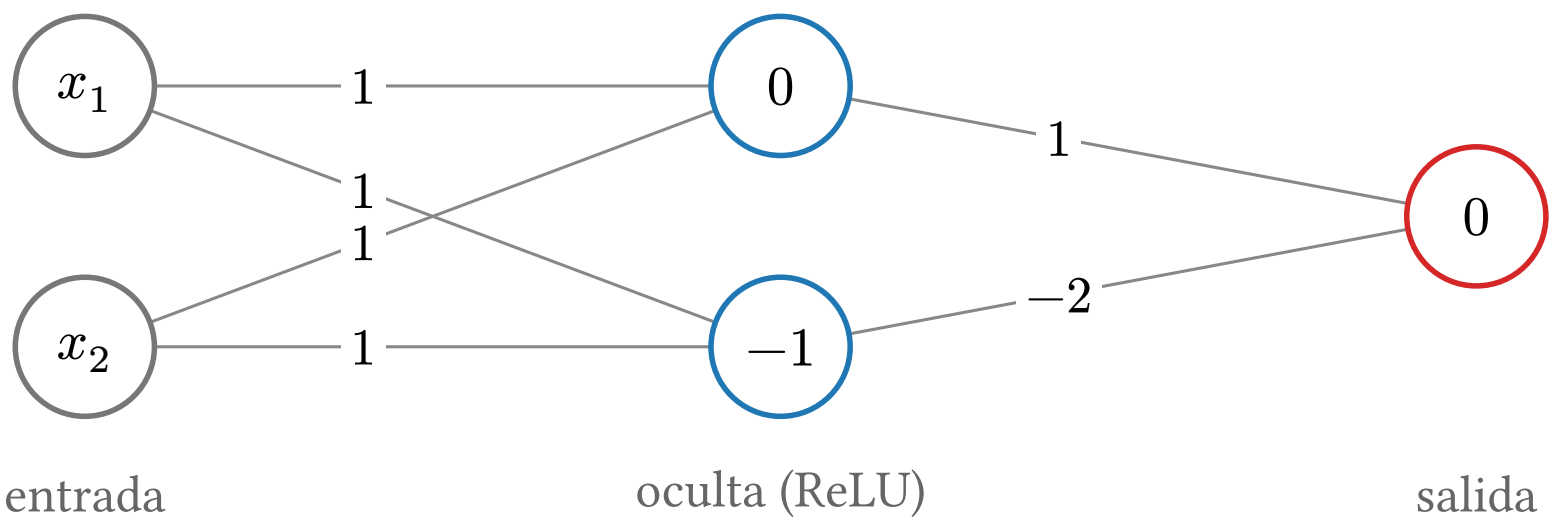


El límite de una neurona: XOR

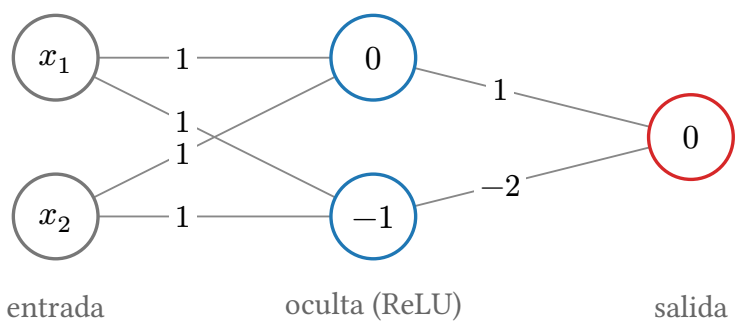
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



Una red que resuelve XOR



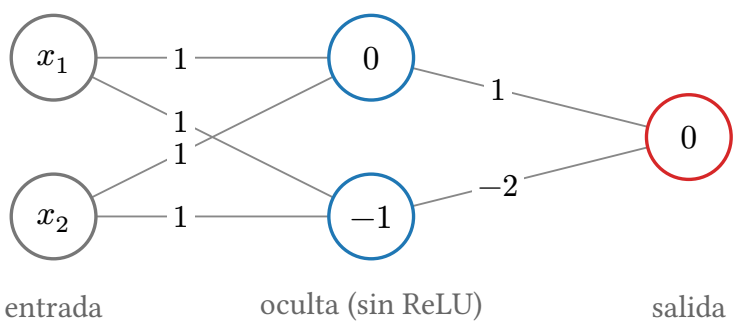
Ejercicio 1



Calcula la predicción de la red para los cuatro ejemplos de XOR y completa la tabla. ¿La red acierta en todos?

x_1	x_2	$x_1 + x_2$	a_1	a_2	$\hat{y}$
0	0				
0	1				
1	0				
1	1				

Ejercicio 2



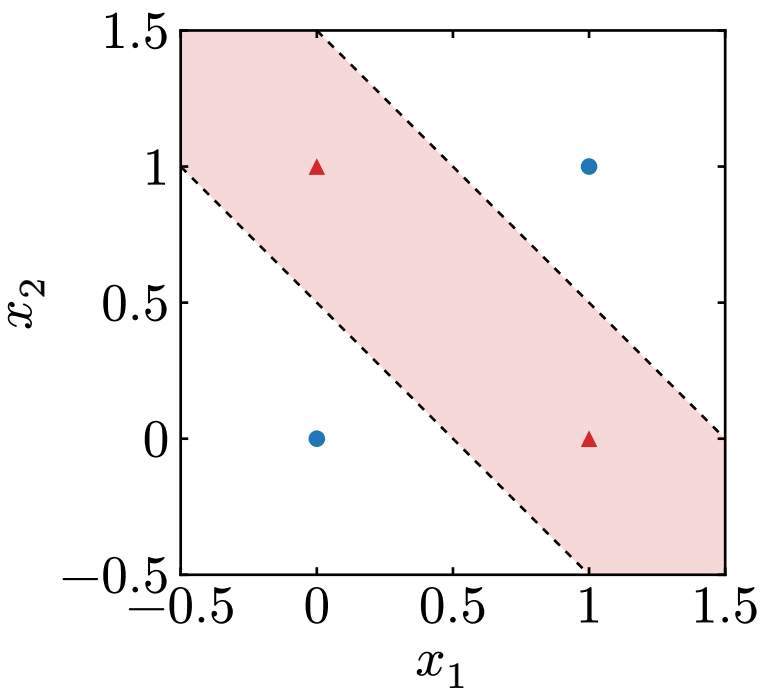
Repite el Ejercicio 1 **sin la ReLU**:

$$a_1 = x_1 + x_2$$
$$a_2 = x_1 + x_2 - 1$$
$$\hat{y} = a_1 - 2a_2$$

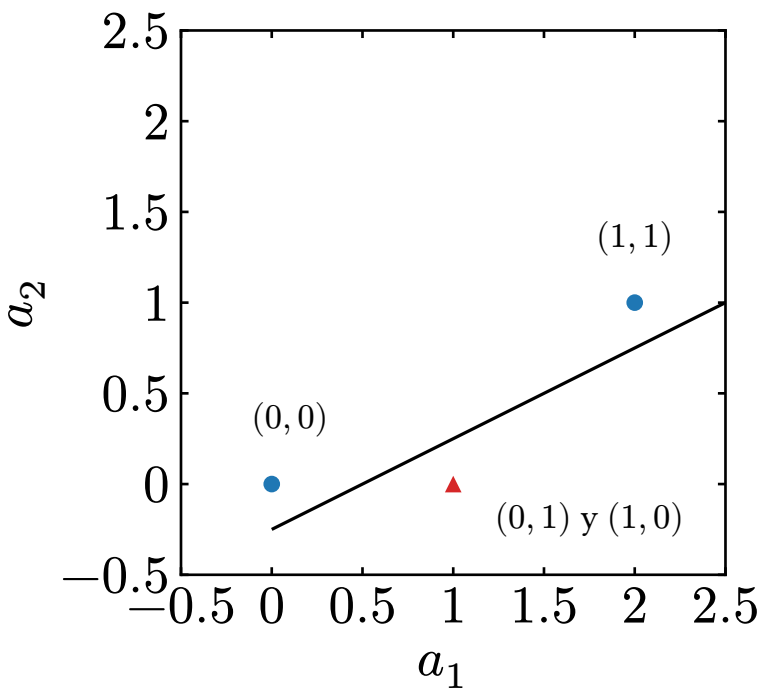
1. Calcula $\hat{y}$ para los cuatro ejemplos. ¿Sigue resolviendo XOR?
2. Sustituye a_1 y a_2 en la fórmula de $\hat{y}$ y simplifica. ¿Qué tipo de función de x_1 y x_2 queda?

La capa oculta transforma los datos

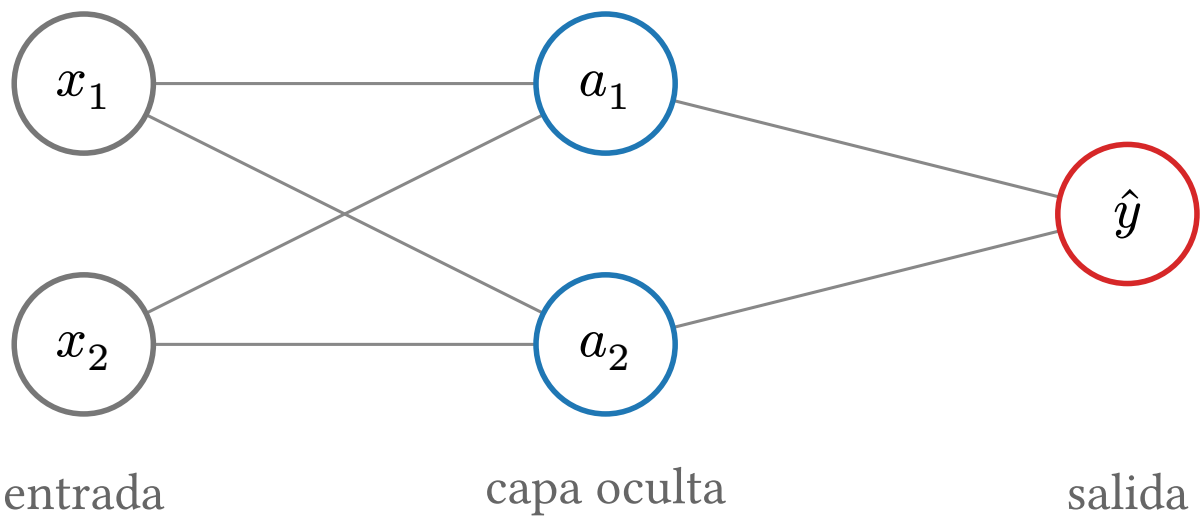
Espacio de las entradas



Espacio de las unidades ocultas

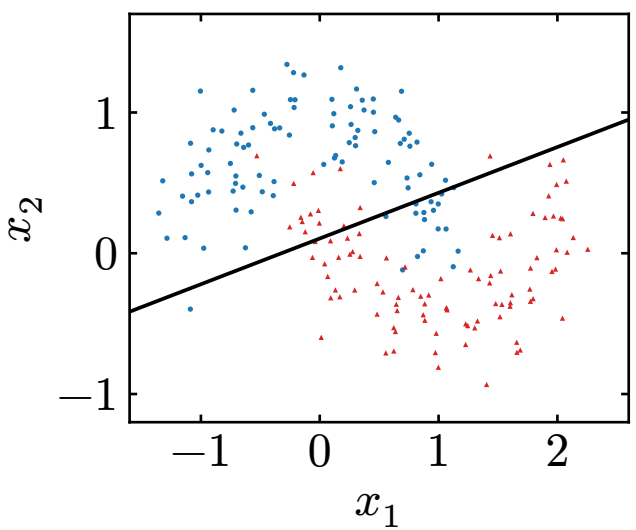


Redes totalmente conectadas



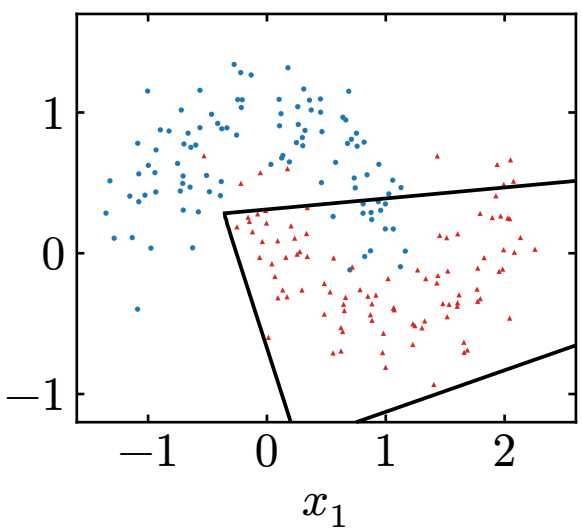
La red aprende las features

Regresión logística



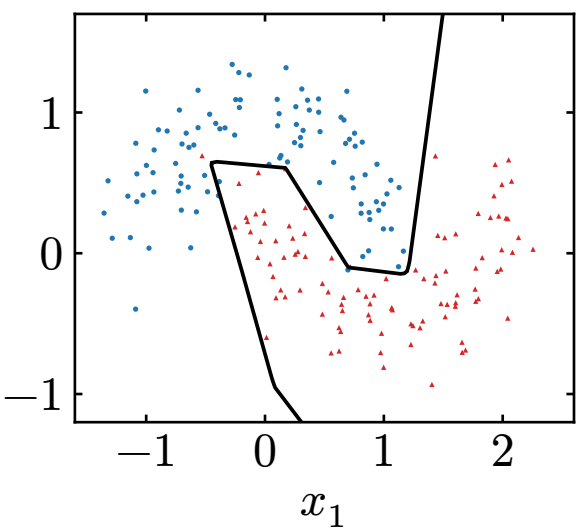
exactitud 86.8%

3 unidades ocultas



exactitud 88.6%

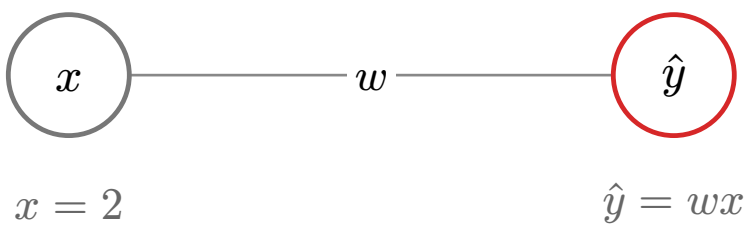
32 unidades ocultas



exactitud 95.9%

clase 0 clase 1

Ejercicio 3



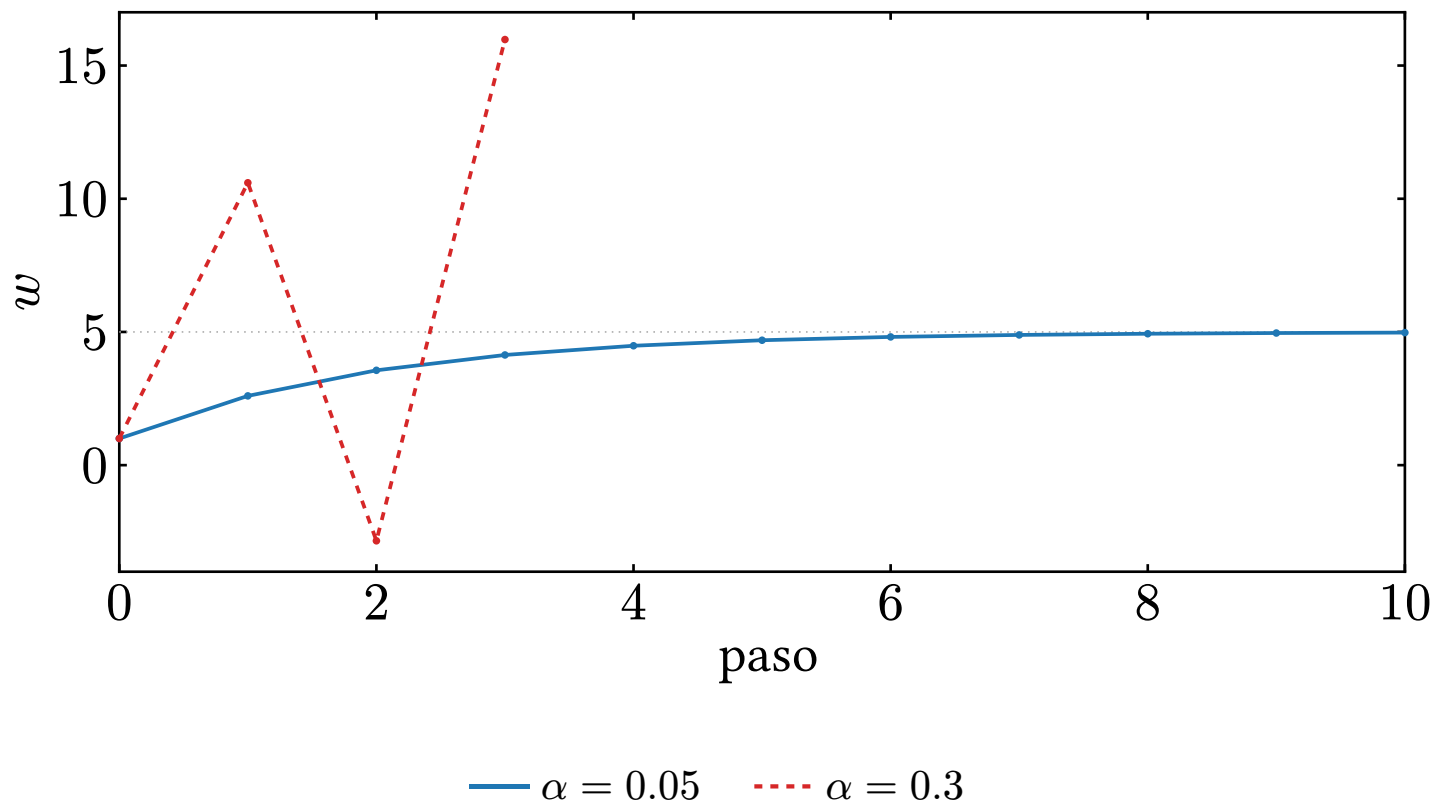
$$y = 10, \quad L = (\hat{y} - y)^2$$
$$\frac{dL}{dw} = 2(\hat{y} - y)x$$

Empezando con $w = 1$ y $\alpha = 0.05$, completa tres pasos de descenso por gradiente.

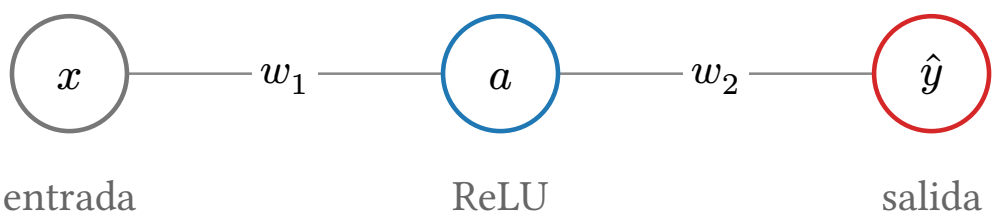
Paso	w	$\hat{y} = 2w$	$L = (\hat{y} - 10)^2$	$\frac{dL}{dw} = 4(\hat{y} - 10)$
0	1			
1				
2				
3				

¿Hacia qué valor se acerca w? ¿Por qué?

Descenso por gradiente: la tasa de aprendizaje



Ejercicio 4



$$a = \text{ReLU}(w_1 x), \quad \hat{y} = w_2 a$$
$$L = (\hat{y} - y)^2$$

Con $x = 2$ y $y = 10$, calcula el forward pass y las dos derivadas $\frac{\partial L}{\partial w_1}$ y $\frac{\partial L}{\partial w_2}$ si empezamos con $w_1 = -1$ y $w_2 = 1$.

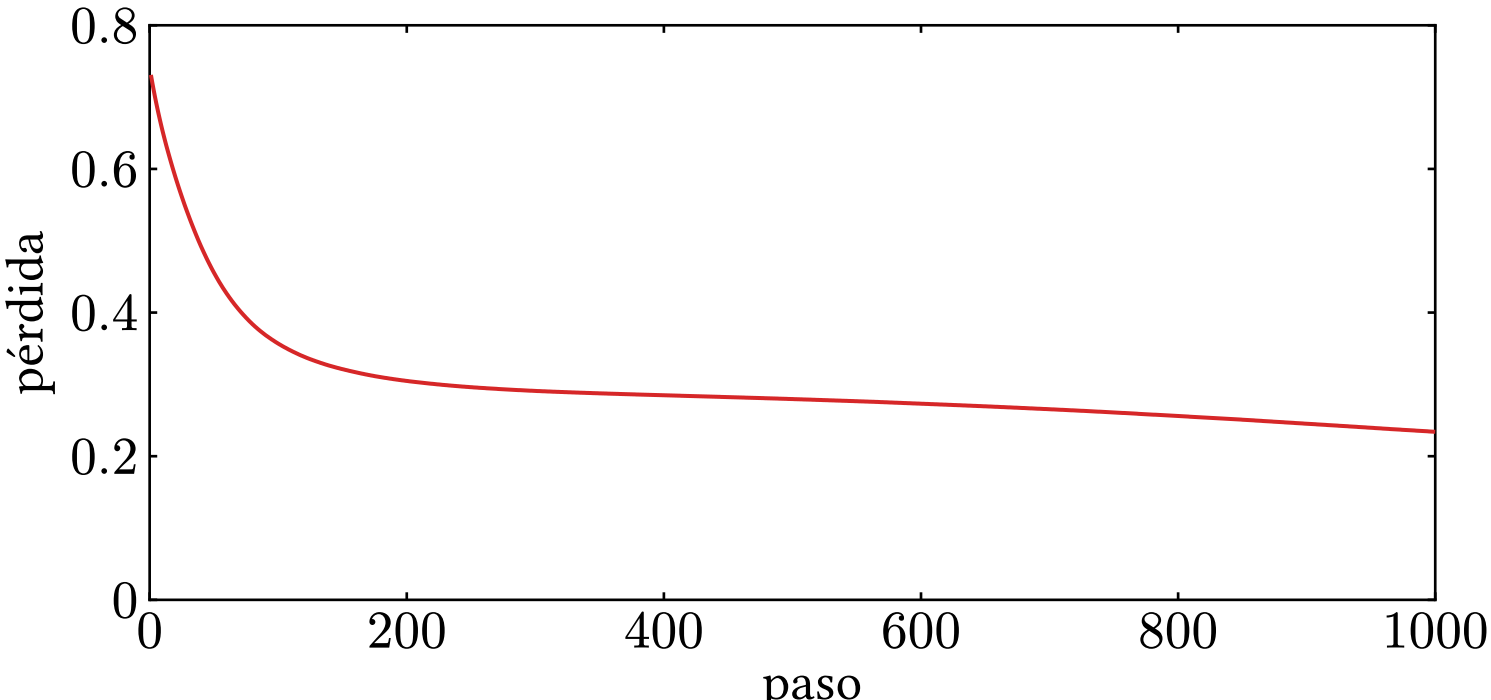
¿Qué pasa con w_1 si damos un paso de descenso por gradiente? ¿Por qué?

Entrenar una red en PyTorch

```
modelo = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 1))
perdida = nn.BCEWithLogitsLoss()
optimizador = torch.optim.SGD(modelo.parameters(), lr=0.1)

for paso in range(1000):
    optimizador.zero_grad()   # 1. borrar los gradientes anteriores
    y_hat = modelo(X)         # 2. forward pass
    L = perdida(y_hat, y)     # 3. calcular la pérdida
    L.backward()              # 4. backpropagation
    optimizador.step()        # 5. un paso de descenso
```

La pérdida durante el entrenamiento



32 unidades ocultas, $\alpha = 0.1$. Exactitud en prueba: 90.4%